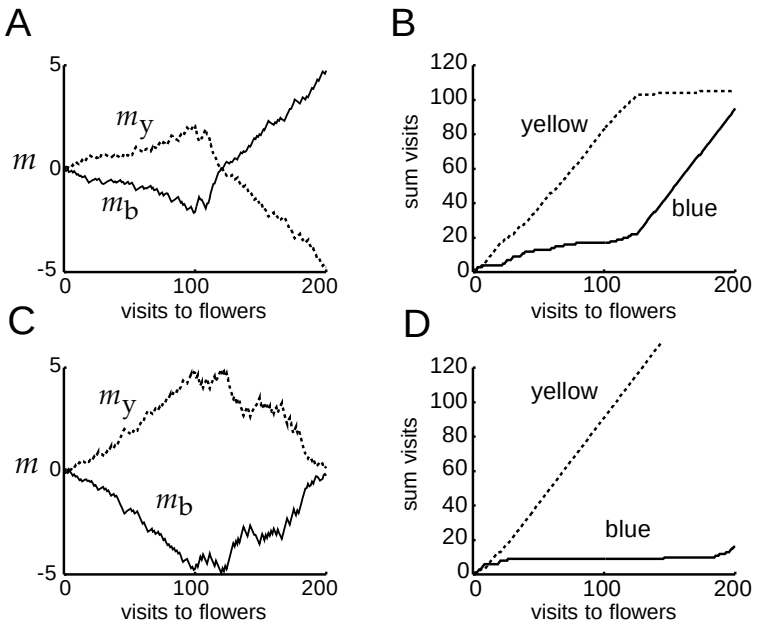

Figure 9.6 The direct actor. The statistics of the delivery of reward are the same as in figure 9.4, and $\epsilon = 0.1$, $\overline{r} = 1.5$, $\beta = 1$. The evolution of the weights and cumulative choices of flower type (with yellow dashed and blue solid) are shown for two sample sessions, one with good performance (A and B) and one with poor performance (C and D).

9.4 Sequential Action Choice

In the previous section, we considered ways that animals might learn to choose actions on the basis of immediate information about the consequences of those actions. A significant complication that arises when reward is based on a sequence of actions is illustrated by the maze task shown in figure 9.7. In this example, a hungry rat has to move through a maze, starting from point A, without retracing its steps. When it reaches one of the shaded boxes, it receives the associated number of food pellets and is removed from the maze. The rat then starts again at A. The task is to optimize the total reward, which in this case entails moving left at A and right at B. It is assumed that the animal starts knowing nothing about the structure of the maze or about the rewards.

If the rat started from point B or point C, it could learn to move right or left (respectively), using the methods of the previous section, because it experiences an immediate consequence of its actions in the delivery or nondelivery of food. The difficulty arises because neither action at the actual starting point, A, leads directly to a reward. For example, if the rat goes left at A and also goes left at B, it has to figure out that the former choice was good but the latter was bad. This is a typical problem in tasks that involve delayed rewards. The reward for going left at A is delayed

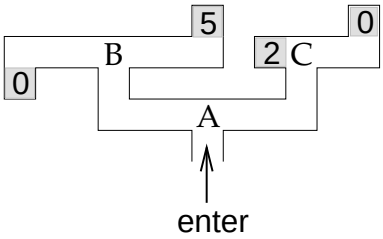

Figure 9.7 The maze task. The rat enters the maze from the bottom and has to move forward. Upon reaching one of the end points (the shaded boxes), it receives the number of food pellets indicated and the trial ends. Decision points are A, B, and C.

until after the rat also goes right at B.

There is an extensive body of theory in engineering, called dynamic programming, as to how systems of any sort can come to select appropriate actions in optimizing control problems similar to (and substantially more complicated than) the maze task. An important method on which we focus is called policy iteration. Our reinforcement learning version of policy iteration maintains and improves a stochastic policy, which determines the actions at each decision point (i.e., left or right turns at A, B, or C) through action values and the softmax distribution of equation 9.12. Policy iteration involves two elements. One, called the critic, uses temporal difference learning to estimate the total future reward that is expected when starting from A, B, or C, when the current policy is followed. The other element, called the actor, maintains and improves the policy. Adjustment of the action values at point A is based on predictions of the expected future rewards associated with points B and C that are provided by the critic. In effect, the rat learns the appropriate action at A, using the same methods of static action choice that allow it to learn the appropriate actions at B and C. However, rather than using an immediate reward as the reinforcement signal, it uses the expectations about future reward that are provided by the critic.

dynamic programming

policy iteration

critic

actor

The Maze Task

As we mentioned when discussing the direct actor, a stochastic policy is a way of assigning a probability distribution over actions (in this case choosing to turn either left or right) to each location (A, B, or C). The location is specified by a variable u that takes the values A, B, or C, and a two-component action value vector $\mathbf{m}(u)$ is associated with each location. The components of the action vector $\mathbf{m}(u)$ control the probability of taking a left or a right turn at u through the softmax distribution of equation 9.12.

The immediate reward provided when action a is taken at location u is written as $r_a(u)$. For the maze of figure 9.7, this takes values 0, 2, or 5, depending on the values of u and a. The predicted future reward expected

at location u is given by $v(u) = w(u)$. This is an estimate of the total award that the rat expects to receive, on average, if it starts at the point u and follows its current policy through to the end of the maze. The average is taken over the stochastic choices of actions specified by the policy. In this case, the expected reward is simply equal to the weight. The learning procedure consists of two separate steps: policy evaluation, in which $w(u)$ is adjusted to improve the predictions of future reward, and policy improvement, in which $\mathbf{m}(u)$ is adjusted to increase the total reward.

Policy Evaluation

In policy evaluation, the rat keeps its policy fixed (i.e., keeps all the $\mathbf{m}(u)$ fixed) and uses temporal difference learning to determine the expected total future reward starting from each location. Suppose that, initially, the rat has no preference for turning left or right, that is, $\mathbf{m}(u) = 0$ for all u, so the probability of left and right turns is $1/2$ at all locations. By inspection of the possible places the rat can go, we find that the values of the states are

$$\begin{aligned} v(\mathrm{B}) &= \frac{1}{2}(0+5) = 2.5\,, \quad v(\mathrm{C}) = \frac{1}{2}(0+2) = 1\,, \quad \text{and} \\ v(\mathrm{A}) &= \frac{1}{2}(v(\mathrm{B}) + v(\mathrm{C})) = 1.75\,. \end{aligned} \tag{9.23}$$

These values are the average total future rewards that will be received during exploration of the maze when actions are chosen using the random policy. The temporal difference learning rule of equation 9.10 can be used to learn them. If the rat chooses action a at location u and ends up at location u', the temporal difference rule modifies the weight $w(u)$ by

critic learning rule

$$w(u) \to w(u) + \epsilon\delta \quad \text{with} \quad \delta = r_a(u) + v(u') - v(u)\,. \tag{9.24}$$

Figure 9.8 shows the result of applying the temporal difference rule to the maze task of figure 9.7. After a fairly short adjustment period, the weights $w(u)$ (and thus the predictions $v(u)$) fluctuate around the correct values for this policy, as given by equation 9.23. The size of the fluctuations could be reduced by making ϵ smaller, but at the expense of increasing the learning time.

In our earlier description of temporal difference learning, we included the possibility that the reward delivery might be stochastic. Here, that stochasticity is the result of a policy that makes use of the information provided by the critic. In the appendix, we discuss a Monte Carlo interpretation of the terms in the temporal difference learning rule that justifies its use.

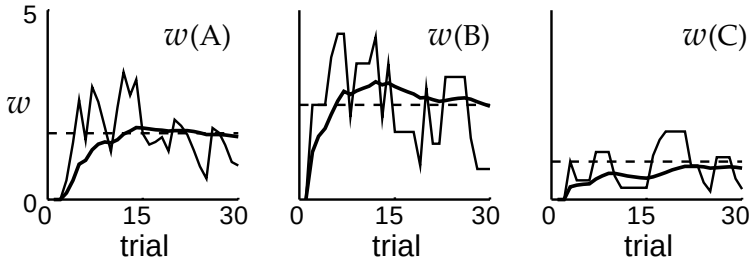

Figure 9.8 Policy evaluation. The thin lines show the course of learning of the weights $w(\mathrm{A})$, $w(\mathrm{B})$, and $w(\mathrm{C})$ over trials through the maze in figure 9.7, using a random unbiased policy ($\mathbf{m}(u) = 0$). Here $\epsilon = 0.5$, so learning is fast but noisy. The dashed lines show the correct weight values from equation 9.23. The thick lines are running averages of the weight values.

Policy Improvement

In policy improvement, the expected total future rewards at the different locations are used as surrogate immediate rewards. Suppose the rat is about to take action a at location u and move to location u'. The expected worth to the rat of that action is the sum of the actual reward received and the rewards that are expected to follow, which is $r_a(u) + v(u')$. For simplicity, we assume that the rat receives the reward for location u at the same time it decides to move on to location u'. The direct actor scheme of equation 9.22 uses the difference $r_a - \bar{r}$ between a sample of the worth of the action (r_a) and a reinforcement comparison term ($\bar{r}$), which might be the average value over all the actions that can be taken. Policy improvement uses $r_a(u) + v(u')$ as the equivalent of the sampled worth of the action, and $v(u)$ as the average value across all actions that can be taken at u. The difference between these is $\delta = r_a(u) + v(u') - v(u)$, which is exactly the same term as in policy evaluation (equation 9.24). The policy improvement or actor learning rule is then *actor learning rule*

$$m_{a'}(u) \to m_{a'}(u) + \epsilon \left(\delta_{aa'} - P[a'; u]\right) \delta \tag{9.25}$$

for all a', where $P[a'; u]$ is the probability of taking action a' at location u given by the softmax distribution of equation 9.12 with action value $m_{a'}(u)$.

To look at this more concretely, consider the temporal difference error starting from location $u = \mathrm{A}$, using the true values of the locations given by equation 9.23 (i.e., assuming that policy evaluation is perfect). Depending on the action, δ takes the two values

$$\begin{aligned} \delta &= 0 + v(\mathrm{B}) - v(\mathrm{A}) = 0.75 && \text{for a left turn} \\ \delta &= 0 + v(\mathrm{C}) - v(\mathrm{A}) = -0.75 && \text{for a right turn.} \end{aligned}$$

The learning rule of equation 9.25 increases the probability that the action with $\delta > 0$ is taken and decreases the probability that the action with $\delta < 0$

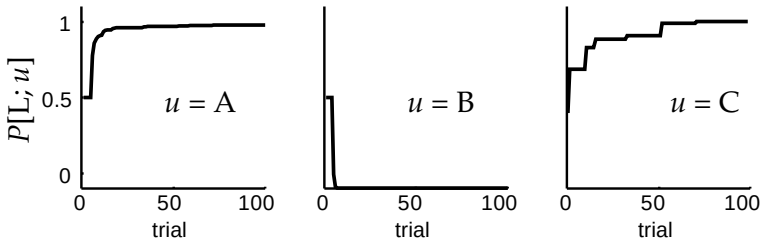

Figure 9.9 Actor-critic learning. The three curves show $P[\mathrm{L}; u]$ for the three starting locations u = A, B, and C in the maze of figure 9.7. These rapidly converge to their optimal values, representing left turns at A and C and a right turn at B. Here, $\epsilon = 0.5$ and $\beta = 1$.

is taken. This increases the chance that the rat makes the correct turn (left) at A in the maze of figure 9.7.

As the policy changes, the values, and therefore the temporal difference terms, change as well. However, because the values of all locations can increase only if we choose better actions at those locations, this form of policy improvement inevitably leads to higher values and better actions. This monotonic improvement (or at least not worsening) of the expected future rewards at all locations is proved formally in the dynamic programming theory of policy iteration for a class of problems called Markov decision problems (which includes the maze task), as discussed in the appendix.

Markov decision problems

Strictly speaking, policy evaluation should be complete before a policy is improved. It is also most straightforward to improve the policy completely before it is re-evaluated. A convenient (though not provably correct) alternative is to interleave partial policy evaluation and policy improvement steps. This is called the actor-critic algorithm. Figure 9.9 shows the result of applying this algorithm to the maze task. The plots show the development over trials of the probability of choosing to go left, $P[\mathrm{L}; u]$, for all three locations. The model rat quickly learns to go left at location A and right at B. Learning at location C is slow because the rat quickly learns that it is not worth going to C at all, so it rarely gets to try the actions there. Thus the algorithm makes an implicit choice of exploration strategy.

actor-critic algorithm

Generalizations of Actor-Critic Learning

The full actor-critic model for solving sequential action tasks includes three generalizations of the maze learner that we have presented. The first involves additional information that may be available at the different locations. If, for example, sensory information is available at a location u, we associate a state vector $\mathbf{u}(u)$ with that location. The vector $\mathbf{u}(u)$ parameterizes whatever information is available at location u that might help the animal decide which action to take. For example, the state vector might represent a scent of food that the rat might detect in the maze task. When a state vector is available, $v(u)$, which is the value at location

state vector $\mathbf{u}$

u, can depend on it. The simplest dependence is provided by the linear form $v(u) = \mathbf{w} \cdot \mathbf{u}(u)$, similar to the input-output relationship used in linear feedforward network models. The learning rule for the critic (equation 9.24) is then generalized to include the information provided by the state vector,

$$\mathbf{w} \to \mathbf{w} + \epsilon \delta \mathbf{u}(u) \,, \tag{9.26}$$

with δ given as in equation 9.24. The maze task we discussed could be formulated in this way by using what is called a unary representation, $\mathbf{u}(\mathrm{A}) = (1, 0, 0)$, $\mathbf{u}(\mathrm{B}) = (0, 1, 0)$, and $\mathbf{u}(\mathrm{C}) = (0, 0, 1)$.

unary representation

We must also modify the actor learning rule to make use of the information provided by the state vector. This is done by generalizing the action value vector $\mathbf{m}$ to a matrix $\mathbf{M}$, called an action matrix. $\mathbf{M}$ has as many columns as there are components of $\mathbf{u}$ and as many rows as there are actions. Given input $\mathbf{u}$, action a is chosen at location u with the softmax probability of equation 9.12, but using component a of the action value vector,

action matrix $\mathbf{M}$

$$\mathbf{m} = \mathbf{M} \cdot \mathbf{u}(u) \quad \text{or} \quad m_a = \sum_b M_{ab} u_b(u) \,. \tag{9.27}$$

In this case, the learning rule 9.25 must be generalized to specify how to change elements of the action matrix when action a is chosen at location u with state vector $\mathbf{u}(u)$, leading to location u'. A rule similar to equation 9.25 is appropriate, except that the change in $\mathbf{M}$ depends on the state vector $\mathbf{u}$,

$$M_{a'b} \to M_{a'b} + \epsilon \left(\delta_{aa'} - P[a'; u]\right) \delta u_b(u) \tag{9.28}$$

three-term covariance rule

for all a', with δ given again as in equation 9.24. This is called a three-term covariance learning rule.

We can speculate about the biophysical significance of the three-term covariance rule by interpreting $\delta_{aa'}$ as the output of cell a' when action a is chosen (which has mean value $P[a'; u]$) and interpreting $\mathbf{u}$ as the input to that cell. Compared with the Hebbian covariance rules studied in chapter 8, learning is gated by a third term, the reinforcement signal δ. It has been suggested that the dorsal striatum, which is part of the basal ganglia, is involved in the selection and sequencing of actions. Terminals of axons projecting from the substantia nigra pars compacta release dopamine onto synapses within the striatum, suggesting that they might play such a gating role. The activity of these dopamine neurons is similar to that of the VTA neurons discussed previously as a possible substrate for δ.

dorsal striatum
basal ganglia

The second generalization is to the case that rewards and punishments received soon after an action are more important than rewards and punishments received later. One natural way to accommodate this is a technique called exponential discounting. In computing the expected future reward, this amounts to multiplying a reward that will be received τ time steps after a given action by a factor γ^τ, where $0 \le \gamma \le 1$ is the discounting factor. The smaller γ, the stronger the effect (i.e., the less important temporally

discounting

distant rewards are). Discounting has a major influence on the optimal behavior in problems for which there are many steps to a goal. Exponential discounting can be accommodated within the temporal difference framework by changing the prediction error δ to

$$\delta = r_a(u) + \gamma v(u') - v(u)\,, \tag{9.29}$$

which is then used in the learning rules of equations 9.26 and 9.28.

In computing the amount to change a weight or action value, we defined the worth of an action as the sum of the immediate reward delivered and the estimate of the future reward arising from the next state. A final generalization of actor-critic learning comes from basing the learning rules on the sum of the next two, three, or more immediate rewards delivered and basing the estimate of the future reward on more temporally distant times within a trial. As in discounting, we can use a factor λ to weight how strongly the expected future rewards from temporally distant points affect learning. Suppose that $\mathbf{u}(t) = \mathbf{u}(u(t))$ is the state vector used at time step t of a trial. Such generalized temporal difference learning can be achieved by computing new state vectors, defined by the recursive relation

$$\tilde{\mathbf{u}}(t) = \tilde{\mathbf{u}}(t-1) + (1-\lambda)(\mathbf{u}(t) - \tilde{\mathbf{u}}(t-1))\,, \tag{9.30}$$

and using them instead of the original state vectors $\mathbf{u}$ in equations 9.26 and 9.28. These new state vectors $\tilde{\mathbf{u}}(t)$ are called stimulus traces, and the resulting learning rule is called the TD(λ) rule. Use of this rule with an appropriate value of λ can significantly speed up learning.

stimulus traces

TD(λ) rule

Learning the Water Maze

As an example of generalized reinforcement learning, we consider the water maze task. This is a navigation problem in which rats are placed in a large pool of milky water and have to swim around until they find a small platform that is submerged slightly below the surface of the water. The opaqueness of the water prevents them from seeing the platform directly, and their natural aversion to water (although they are competent swimmers) motivates them to find the platform. After several trials, the rats learn the location of the platform and swim directly to it when placed in the water.

Figure 9.10A shows the structure of the model, with the state vector $\mathbf{u}$ providing input to the critic and a collection of eight possible actions for the actor, which are expressed as compass directions. The components of $\mathbf{u}$ represent the activity of hippocampal place cells (which are discussed in chapters 1 and 8). Figure 9.10B shows the activation of one of the input units as a function of spatial position in the pool. The activity, like that of a place cell, is spatially restricted.

During training, each trial consists of starting the model rat from a random location at the outside of the maze and letting it run until it finds

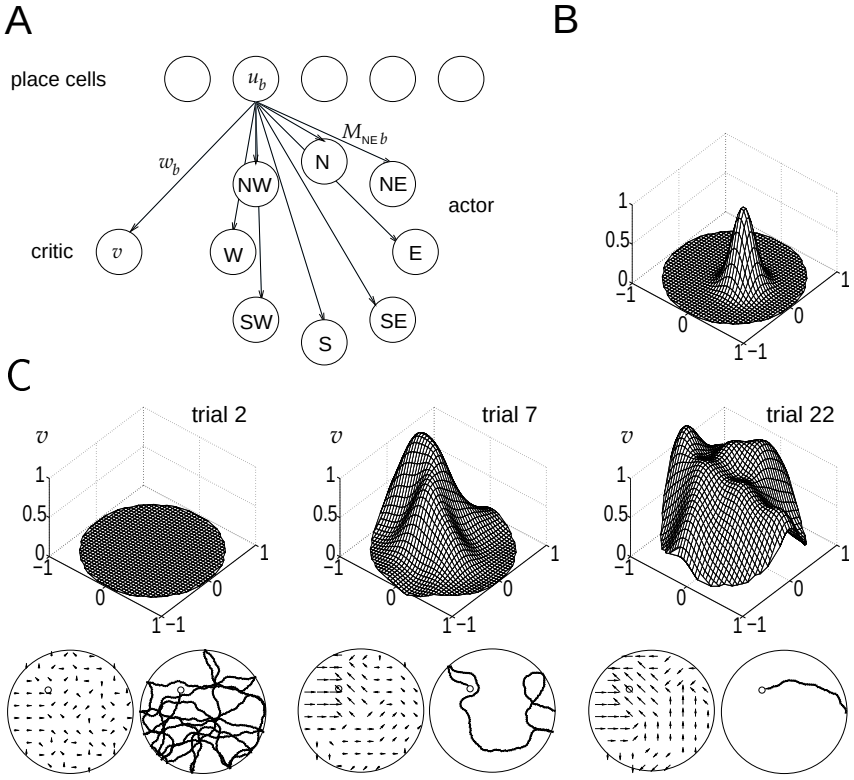

Figure 9.10 Reinforcement learning model of a rat solving a simple water maze task in a 2 m diameter circular pool. (A) There are 493 place cell inputs and 8 actions. The rat moves at 0.3 m/s and reflects off the walls of the maze if it hits them. (B) Gaussian place field for a single input cell with width 0.16 m. The centers of the place fields for different cells are uniformly distributed across the pool. (C) Upper: The development of the value function v as a function of the location in the pool over the first 22 trials, starting from $v = 0$ everywhere . Lower arrow plots: The action with the highest probability for each location in the maze. Lower path plots: Actual paths taken by the model rat from random starting points to the platform, indicated by a small circle. A slight modification of the actor learning rule was used to enforce generalization between spatially similar actions. (Adapted from Foster et al., 2000.)

the platform indicated by a small circle in the lower part of figure 9.10C. At that point a reward of 1 is provided. The reward is discounted with $\gamma = 0.9975$ to model the incentive for the rat to find the goal as quickly as possible. Figure 9.10C indicates the course of learning (trials 2, 7, and 22) of the expected future reward as a function of location (upper figures) and the policy (lower figures with arrows). The lower figures also show sample paths taken by the rat (lower figures with wiggly lines). The final value function (at trial 22) is rather inaccurate, but nevertheless the policy learned is broadly correct, and the paths to the platform are quite short and direct.

Judged by measures such as path length, initial learning proceeds in the

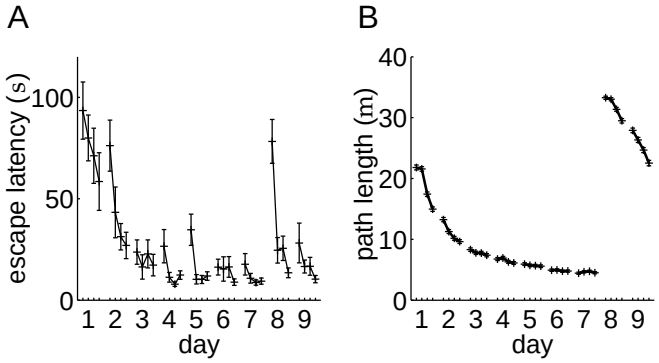

Figure 9.11 Comparison of rats and the model in the water maze task. (A) Average latencies of 12 rats in getting to a fixed platform in the water maze, using four trials per day. On the eighth day, the platform was moved to a new location, which is called reversal. (B) Average path length from 1000 simulations of the model performing the same task. Initial learning matches that of the rats, but performance is worse following reversal. (Adapted from Foster et al., 2000.)

model in a manner comparable to that of actual rats. Figure 9.11A shows the average performance of 12 real rats performing four trials per day in the water maze. The rats swam to a platform at a fixed location, starting from randomly chosen initial locations. The performance of the rats rapidly improves and levels off by about the sixth day. When the platform is moved on the eighth day, in what is called reversal training, the initial latency is long, because the rats search near the old platform position. However, they rapidly learn the new location. Figure 9.11B shows the performance of the model on the same task (though judged by path lengths rather than latencies). Initial learning is equally quick, with near perfect paths by the sixth day. However, performance during reversal training is poor, because the model has trouble forgetting the previous location of the platform. The rats are clearly better at handling this transition. Nevertheless the model shows something of the power of a primitive, but general, learning method.

9.5 Chapter Summary

We discussed reinforcement learning models for classical and instrumental conditioning, interpreting the former in terms of learning predictions about total future rewards and the latter in terms of optimization of those rewards. We introduced the Rescorla-Wagner or delta learning rule for classical conditioning, together with its temporal difference extension, and indirect and direct actor rules for instrumental conditioning given immediate rewards. Finally, we presented the actor-critic version of the dynamic programming technique of policy iteration, evaluating policies using temporal difference learning and improving them using the direct actor learning rule, based on surrogate immediate rewards from the evaluation step.